The Complete AI & LLM Workflow Guide

Vendors, models, skills, MCP, agents, Claude Code, folder structures, token optimization, and cadence

A practical handbook for working with AI in 2026

Information current as of May 2026

Contents

1. The AI landscape today

Three+1 dominant players

Anthropic (Claude), **OpenAI (ChatGPT / GPT)**, and **Google (Gemini)** hold the high ground on capability. **xAI (Grok)** competes aggressively on price. Open-source frontier: **Meta Llama**, **Mistral**, **DeepSeek**, **Qwen**. Pick by use case, not brand.

The major vendors and their flagships

Vendor	Chat product	API flagship	Consumer tiers	Strengths
Anthropic	Claude (claude.ai)	Claude Opus 4.7	Free / Pro $20 / Max / Team	Safety, agents, long context, code
OpenAI	ChatGPT	GPT-5.5 / GPT-5.5 Pro	Free / Plus $20 / Pro $200 / Team / Enterprise	Ecosystem, plugins, image gen (DALL-E)
Google	Gemini app	Gemini 3.1 Pro / 3.5	Free / AI Pro / AI Ultra	Multimodal native, 2M context, Workspace
xAI	Grok (X)	Grok 4.1	Free w/X / Premium / Heavy	Cheapest API, real-time X data
Meta	Meta AI (WhatsApp/IG)	Llama 4 (open weights)	Free in apps	Open-source, self-host, fine-tune
Mistral	Le Chat	Mistral Large 3	Free / Pro	EU sovereign, open weights
DeepSeek	DeepSeek chat	DeepSeek V3 / R1	Free	Strong reasoning, very low cost
Microsoft	Copilot	GPT-based, custom routing	Free / Pro $20 / M365 Copilot $30/seat	Office integration
Perplexity	Perplexity.ai	Routes multiple models	Free / Pro $20	Web-search-first answers w/ citations

Product surfaces you'll encounter

- **Chat UI** — the web/app interface (claude.ai, chatgpt.com, gemini.google.com). Quick conversations, file uploads, image generation, web search.
- **API** — REST endpoints for developers. You pay per token. All vendors expose one.
- **CLI / Agent tools** — Claude Code, OpenAI Codex CLI, Gemini CLI, Cursor, Windsurf, Aider -- terminal-native coding agents.
- **IDE plugins** — GitHub Copilot, Cursor, Continue.dev, Cody. Inline completions and chat inside VS Code / JetBrains.
- **Workflow platforms** — n8n, Zapier, Make, LangFlow -- no-code orchestration of LLM calls with other apps.
- **Cloud platforms** — AWS Bedrock, Google Vertex AI, Azure OpenAI -- enterprise hosting with private networking, BYOK, compliance.

2. Models & pricing -- the master table

How to read API pricing

All prices are **per 1 million tokens (MTok)**, billed separately for input and output. Output is typically 4-6x more expensive than input. A token is roughly 4 characters of English text (~0.75 words). 1 MTok ≈ 750,000 words ≈ 3,000 typical book pages.

Current-generation flagship API pricing (May 2026)

Model	Input $/MTok	Output $/MTok	Context	Notes
Claude Opus 4.7	$5.00	$25.00	1M	Most capable Claude; same price as 4.6
Claude Sonnet 4.6	$3.00	$15.00	1M	Best balance of price/quality
Claude Haiku 4.5	$1.00	$5.00	200K	Cheapest current Claude, fast
GPT-5.5 (standard)	$5.00	$30.00	400K	Newer OpenAI base model
GPT-5.2	$1.75	$14.00	256K	Mid-tier OpenAI
GPT-5.2 mini	~$0.40	~$3.20	256K	Cheap OpenAI
Gemini 3.1 Pro	$2.00	$12.00	2M	Above 200K context: $4 / $18
Gemini 3 Flash	$0.50	$3.00	1M	Cheap and fast Google option
Gemini 3.1 Flash-Lite	$0.25	~$1.50	1M	Cheapest Google tier
Grok 4.1	$0.20	$0.50	256K	Cheapest of any flagship
DeepSeek V3	~$0.27	~$1.10	128K	Strong open-weight model

Discount mechanisms (apply across vendors)

Mechanism	Typical savings	When it applies
Prompt caching	90% off cached input	Repeated system prompts / RAG docs across calls
Batch API	50% off	Non-realtime jobs OK to finish within 24h
Long-context surcharge	+50-100% above threshold	Some vendors (Gemini >200K, others)
Volume / committed-use	10-30%	Enterprise contracts, AWS Bedrock, Vertex
Open-source self-host	Effectively 0$ token cost	You pay for GPUs and ops instead

Consumer (subscription) tiers

Tier	Anthropic Claude	OpenAI ChatGPT	Google Gemini
Free	Limited Sonnet usage	Limited GPT-5	Limited Gemini 3 Flash
~$20/mo	Pro	Plus	AI Pro
~$30/seat	Team	Team	Workspace add-on
~$100-200	Max ($100-$200)	Pro ($200)	AI Ultra (~$125)
Enterprise	Enterprise	Enterprise	Vertex AI / Workspace

■ Prices change quarterly

Treat these as a snapshot. Anthropic, OpenAI, and Google all repriced in the last 6 months. Before committing to a tier, check the vendor's current pricing page. The **ratio** between tiers tends to stay stable even when absolute numbers shift.

3. AI / NLP / LLM -- the underlying tech

The stack in one paragraph

Text becomes **tokens**. Tokens become high-dimensional **embeddings**. A **transformer** (stacked attention layers) predicts the next token. Train it on massive text ⇒ **LLM**. Apply RLHF for alignment. At inference, feed it your prompt, sample tokens until done. Add **tools** (function calls) and you have an **agent**. Add **retrieval** (vector DB lookup) and you have **RAG**. Add **system prompts + skills + memory** and you have a useful assistant.

Concept map

Term	What it really is	When you'll hear it
AI	Umbrella term: anything machine-learned that mimics intelligence	Marketing, news
ML	Machine learning: algorithms that improve from data	Technical, broad
DL	Deep learning: ML using deep neural networks	Research papers
NLP	Natural language processing -- old field, now dominated by LLMs	Academic / classical
LLM	Large Language Model -- transformer trained on text	Most current AI discussion
GenAI	Generative AI: produces text/image/audio/video	Business speak
Foundation model	Pretrained model used as base for many tasks	Industry term
Token	Subword unit; the atom of LLM input/output	Pricing, context
Embedding	Vector representation of text (semantic similarity)	Search, RAG
Context window	Max tokens the model can see in one call	Capacity discussion
Prompt	The input you send to the model	Daily use
System prompt	Instructions visible across the whole conversation	API / configuration
RAG	Retrieval-Augmented Generation: look up docs, stuff into prompt	Knowledge bases
Fine-tuning	Further training on your data	Customization
RLHF / DPO	Aligning models to human preferences	Training papers
Tool use / function calling	Model calls external code/APIs	Agents
MCP	Standardized way to expose tools to LLMs	Integrations
Agent	LLM that plans + uses tools + iterates	2024+ buzzword
Multimodal	Handles text + image + audio + video	Modern flagships
Inference	Running a trained model to get output	Cost / latency
Training	Building the model from data	One-time, expensive
Temperature	Randomness knob: 0 = deterministic, 1+ = creative	Tuning
Hallucination	Model confidently states something false	Reliability

How text becomes capability (the pipeline)

```
1. Raw text:  'The memory wall slows AI inference.'
2. Tokenize:  ['The', ' memory', ' wall', ' slows', ' AI', ' inference', '.']
3. Embed:     each token -> vector of ~12,288 dims (Opus-scale)
4. Transform: 80+ attention layers compute context-aware representations
5. Predict:   output a probability distribution over the next token
6. Sample:    pick a token (greedy / top-p / temperature)
7. Repeat:    feed the new sequence back, generate next token, until
```

RAG vs. fine-tuning vs. long context: when to use what

Technique	Use when	Strengths	Drawbacks
Just use context	Docs fit in window	Simplest, fast, fresh	Token cost; context limit
RAG (retrieval)	Knowledge base too big for context	Cheap per query, updatable	Retrieval quality is hard
Fine-tuning	Output style / domain language	Bakes in behavior	Slow, expensive, stale
Long context (>200K)	Whole codebase / book in one shot	No retrieval to maintain	Expensive per call
Prompt caching	Same prefix used repeatedly	90% cost cut on prefix	Vendor-specific

4. Claude vs the competition

Pick-by-task summary

For **code**, Claude (Opus 4.7 / Sonnet 4.6) and GPT-5.5 lead. For **long-context** tasks (whole codebases, large PDFs), Gemini 3.1 Pro's 2M window is unmatched. For **creative writing**, Claude is widely preferred. For **multimodal** (video + audio), Gemini wins. For **cheap high-volume routing**, Haiku 4.5, GPT-5.2 mini, or Grok 4.1.

Capability comparison by task

Task	Best (2026)	Strong alternative	Cheap option
Long-form code, agentic	Claude Opus 4.7	GPT-5.5 Pro	Sonnet 4.6
IDE inline completion	Cursor (Claude/GPT)	Copilot (GPT)	Continue.dev + Haiku/Mini
Writing / editing prose	Claude Sonnet 4.6	Claude Opus 4.7	GPT-5.2
Research / web answers	Perplexity Pro	ChatGPT search	Gemini AI Pro
Reading large PDFs	Gemini 3.1 Pro (2M)	Claude Opus 4.7 (1M)	Gemini 3 Flash
Image generation	GPT Image 1.5	Gemini Nano Banana Pro	Stable Diffusion (self-host)
Video / audio analysis	Gemini 3.1 Pro	GPT-5.5 (some)	(Limited cheap options)
Math / reasoning	Claude Opus 4.7 (extended thinking)	GPT-5.5 Pro	DeepSeek R1
High-volume routing / triage	Claude Haiku 4.5	GPT-5.2 mini	Grok 4.1
Spreadsheet / data	Claude (with Code Interp.)	GPT (Code Interp.)	Local Python + Sonnet
Voice assistant	ChatGPT Advanced Voice	Gemini Live	(open-source: faster-whisper)

Task	Best (2026)	Strong alternative	Cheap option
Open-source / self-host	Llama 4	Mistral Large 3	DeepSeek V3

Honest tradeoffs

Claude pros	Claude cons
Outstanding code agent (Claude Code CLI)	More expensive per token than Gemini/Grok
Strong long-form writing	Image gen only via partner integrations
Skills + MCP first-class	Smaller plugin/ecosystem than OpenAI
Reliable instruction-following	No native voice mode (yet, in Claude.ai)
Best-in-class refusal/safety	Sometimes over-cautious for edgy creative work

GPT pros	GPT cons
Largest plugin/GPT-store ecosystem	Pricing rose with GPT-5.5
DALL-E image gen built-in	Less transparent about reasoning
Voice mode is mature	Less reliable at agentic coding than Claude
Strong reasoning models (o-series)	Context smaller than Gemini

Gemini pros	Gemini cons
2M context window (huge advantage)	Pro now paid-only (no free tier)
Native multimodal video/audio	Less mature agentic coding
Tight Google Workspace integration	Some API surface still in flux
Imagen for high-quality images	Less developer mindshare than OpenAI

5. Claude Code CLI -- deep dive

What it is

Claude Code is Anthropic's **terminal-native coding agent**. You run claude in your project folder; it reads files, edits code, runs commands, and iterates. It uses Claude Opus / Sonnet under the hood and integrates with skills, MCP servers, and your local shell.

Installation (Node.js required)

```
# Install once, globally
npm install -g @anthropic-ai/claude-code

# Authenticate (one-time browser flow)
claude login

# Start in current directory
cd ~/projects/my-app
claude
```

Daily commands you should know

Command (inside claude)	What it does
/help	Show all built-in commands

Command (inside claude)	What it does
/model	Switch between Opus / Sonnet / Haiku
/clear	Reset the conversation context
/compact	Summarize history to save tokens
/init	Generate a CLAUDE.md project file
/permissions	Allow/deny file/network operations
/mcp	List and manage MCP server connections
/skills	List available skills, enable/disable
/cost	Show this session's token usage and $ spent
/agents	Spawn / manage sub-agents
@filename	Reference a specific file in your prompt
!command	Run a shell command and feed output to Claude

The CLAUDE.md file

Put a CLAUDE.md at your project root. Claude reads it at session start. Use it for project-wide context that should always be available:

```
# Project: my-app

## What this is
Customer dashboard for ACME Corp. React 18 + TS, Python FastAPI backend.

## Build / test commands
- frontend: cd web && pnpm dev | pnpm test
- backend:  cd api && uv run pytest

## Conventions
- All API routes start with /api/v1
- Use TypeScript strict mode
- No new dependencies without asking first

## Files NOT to touch
- /infra (Terraform; owned by DevOps)
- /vendor (third-party)
```

■ Keep CLAUDE.md short

Every token in CLAUDE.md is loaded on EVERY turn. Target 100-200 lines. Push details into referenced docs (referenced by path) and let Claude load them on demand.

Typical workflows

- **Investigate a bug** — claude -> 'There's a 500 on POST /api/users when email contains +. Find and fix.' Claude reads files, makes a hypothesis, applies a fix, runs tests.
- **Code review** — 'Review the changes in the last commit. Flag any issues.'
- **Refactor** — 'Extract the auth logic in routes/* into middleware/auth.ts. Update callers. Run tests.'
- **Tests for legacy code** — 'There are no tests for services/billing.ts. Write pytest tests covering the main paths and edge cases.'
- **Doc generation** — 'Generate a README section explaining how to deploy this service. Look at infra/ and the Dockerfile.'

■ **Auto-approve carefully**

claude --dangerously-skip-permissions lets the agent run shell commands without asking. Only use in disposable sandboxes (Docker, scratch repos). For your real codebase, leave permissions ON and approve each file edit / command.

6. The Skills system

Skills in one paragraph

A **Skill** is a folder with a SKILL.md file plus optional scripts, references, and templates. Claude reads the skill's metadata (name + description) at startup, but only loads the full SKILL.md and supporting files **when the skill is triggered** by something in the conversation. This is **progressive disclosure** -- saving context window. You can ship skills with Claude Code, Claude.ai, or the API.

Anatomy of a skill folder

```
~/.claude/skills/
  excel-reports/
    SKILL.md          <- REQUIRED, ~50-200 lines, the only entry point
    references/       <- loaded on demand by Claude when needed
      formulas.md
      pivot-templates.md
      troubleshooting.md
    scripts/          <- executable Python / bash Claude can run
      validate.py
      build_pivot.py
    assets/           <- static files (templates, schemas)
      report-template.xlsx
      branding.png
    examples/         <- worked examples Claude can study
      monthly-sales.md
```

What goes in SKILL.md

```
---
name: excel-reports
description: Use this when the user asks to create, edit, or analyze
  Excel reports, especially monthly sales / KPI / pivot reports.
  Triggers on words like 'spreadsheet', 'pivot table', '.xlsx',
  'monthly report'.
allowed-tools: [bash, python, view, create_file, str_replace]
---

# Excel reports

## When to use this skill
Any time the deliverable is an .xlsx with formatted tables / pivots /
charts. Not for CSVs (use the data-analysis skill).

## Workflow
1. Ask the user for: data source, columns, target audience
2. Read references/formulas.md if formulas are needed
3. Use scripts/build_pivot.py to scaffold the workbook
4. Add formatting per assets/report-template.xlsx
5. Save to /mnt/user-data/outputs/ and present the file

## Gotchas
- Do not use openpyxl's pandas convenience; it loses styling
- Always validate via scripts/validate.py before presenting
```

Skill scopes: where to put them

Scope	Path	Who sees it
Personal	~/.claude/skills/	You, across all your projects
Project	/.claude/skills/	Anyone working in this repo
Plugin	Inside an installed plugin	Everyone who installs the plugin
Anthropic built-in	(bundled)	All Claude users

Writing effective skills (best practices)

- **Single responsibility** — Each skill does ONE thing. 'Excel reports' is good; 'all office files' is too broad.
- **Description triggers discovery** — The description field is what Claude sees first. Mention the file types, the verbs, and the trigger phrases.
- **Process in SKILL.md, context in references/** — SKILL.md = numbered steps and decision logic. Background info, lookup tables, deep examples go in references/ and are loaded on demand.
- **Keep SKILL.md under 200 lines** — Above that, you waste context. Move anything specific to references/.
- **Be explicit about when NOT to use it** — Add a 'When NOT to use' section so Claude doesn't grab it for adjacent tasks.
- **Include worked examples** — One realistic end-to-end example is worth ten paragraphs of theory.
- **Test the skill** — Use a fresh conversation. Make a request without mentioning the skill name. Does Claude pick it up?

Reuse & versioning

- **Git-track personal skills** — ~/.claude/skills is just a folder of markdown -- put it in a git repo. You can share with teammates, roll back changes, branch experiments.
- **Version inside the skill** — Add a '## Changelog' section at the bottom of SKILL.md. Easier than chasing git history.
- **Skill packs / plugins** — Group related skills into a plugin folder you can install with one command. Useful for company-wide standards.
- **Update by editing SKILL.md** — Restart your Claude session to pick up changes; in Claude Code, /skills reload.

■ How to create a skill: ask Claude itself

In Claude Code or Claude.ai with Code Execution: 'Create a skill called *xyz* for [task]. Follow Anthropic's progressive-disclosure best practices.' Claude will scaffold the folder, write SKILL.md, and even test it. The built-in 'skill-creator' skill exists exactly for this.

■ Skill discovery scales sub-linearly

With 5 skills, Claude picks the right one ~95% of the time. With 50+ overlapping skills, it can grab the wrong one. Curate ruthlessly. Delete skills you haven't used in a month. Keep descriptions distinctive.

7. MCP -- Model Context Protocol

MCP in one paragraph

MCP is the USB-C port for AI. An open protocol (Anthropic, late 2024; now adopted by OpenAI, Google, Cursor, and 200+ tools) that standardizes how an LLM connects to external services -- GitHub, Slack, Postgres, your filesystem, Notion, Stripe. Before MCP: every model had its own integration format (N×M problem). With MCP: build one server per tool, one client per host, every combination works.

Architecture in plain words

```
  [ You / Claude / GPT / Gemini ]                  <- the HOST
          |
          |  MCP client (built into the host)
          |
  =========  MCP protocol (JSON-RPC over stdio or HTTP)  =========
          |                  |                        |
    [GitHub MCP]      [Postgres MCP]          [Filesystem MCP]
    server                server                  server
          |                  |                        |
    GitHub API        your database            your local files
```

What MCP servers exist (sample of 200+)

Category	Notable MCP servers
Code / DevOps	GitHub, GitLab, Linear, Jira, Docker, Kubernetes, Sentry
Databases	Postgres, MySQL, MongoDB, ClickHouse, Snowflake, SQLite
Vector / search	Chroma, Pinecone, Weaviate, Qdrant, Elasticsearch
Productivity	Google Drive, Gmail, Notion, Slack, Microsoft Graph
Browser / scraping	Playwright, Puppeteer, Brave Search, Tavily
Files / local	Filesystem, SQLite, Memory (long-term), Git
Cloud	AWS Bedrock AgentCore, Cloudflare, GCP, Azure
Business	Stripe, Salesforce, HubSpot, PandaDoc, Figma
Automation	n8n, Zapier, Make

Configuring MCP in Claude Code

```
# Add an MCP server (interactive)
claude mcp add

# Or edit ~/.claude.json manually:
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "ghp_..." }
    },
    "postgres": {
      "command": "uvx",
      "args": ["mcp-server-postgres",
               "postgresql://user:pw@localhost/db"]
    }
```

```
  }
}

# In a Claude session, list connected servers:
/mcp
```

MCP vs. Skills vs. function calling -- when to use what

Approach	Best for	Pros	Cons
Skill	Repeated workflows / procedures	Just markdown, no infra	Stays inside the model context
MCP server	Connecting to external services	Standard, reusable across LLMs	Setup, auth, ops
Function calling (raw)	Custom in-app tools	Total control	One-off, model-specific
Plain shell / bash	Local commands during a session	Zero setup	Not portable

Managing MCP going forward (practical advice)

- **Start with 3-5 servers max** — GitHub + Filesystem + your primary DB covers 80% of needs.
- **Use scoped tokens** — Never give an MCP server a full-access token. GitHub: fine-grained PAT; Postgres: read-only role.
- **Audit periodically** — /mcp in Claude Code shows what's active. Review every month; remove what you don't use.
- **Pin versions in production** — npx -y @scope/server@1.4.2 not @latest. Servers update; behavior changes.
- **Sandbox new servers** — Try unfamiliar MCPs in a disposable folder/repo first. Some have broad filesystem access.
- **Prefer official / well-starred servers** — Community quality varies. Anthropic, OpenAI, GitHub, Cloudflare maintain trustworthy refs.

■ Prompt injection via MCP

A malicious or compromised MCP server can inject instructions into Claude's context. Treat MCP output as untrusted data, the same way you'd treat user-supplied HTML. Don't connect random unknown MCPs.

8. AI Agents -- frameworks, patterns, choices

Agent in one sentence

An **agent** is an LLM that runs in a loop: read a goal → pick a tool → execute → observe → repeat until done. The complexity is in **tool design**, **error handling**, and **knowing when to stop**.

The four agent shapes

Shape	Description	Example
Single-shot	One prompt, one answer, no tools	Summarize this PDF
Tool-using	LLM + a fixed set of functions/MCPs	Claude Code editing files
Multi-step (ReAct)	Plan -> act -> observe -> replan	Research a topic with web search

Shape	Description	Example
Multi-agent	Several LLMs / personas talking to each other	PM agent + dev agent + QA agent

Agent frameworks to know

Framework	Language	Sweet spot	Notes
Claude Code	(CLI, any)	Coding tasks in the terminal	By Anthropic
OpenAI Agents SDK	Python/JS	GPT-based agents	By OpenAI
LangChain / LangGraph	Python/JS	Complex multi-step workflows	Mature, big
LlamaIndex	Python/JS	RAG-first agents	Best-in-class retrieval
CrewAI	Python	Multi-agent role-play	Easy to start
AutoGen (Microsoft)	Python	Multi-agent conversations	Research-friendly
Pydantic AI	Python	Type-safe agent code	Newer, clean API
Mastra	TypeScript	Production TS agents	Edge-friendly
Cursor / Windsurf	(IDE)	Coding inside an editor	Hybrid IDE + agent

Agent design principles (what actually works)

- **Narrow the toolset** — 5-10 well-named tools beats 50. The model spends less reasoning on tool selection.
- **One tool = one verb** — read_file, send_email, list_users. Avoid swiss-army tools.
- **Cheap models for routing, expensive for thinking** — Haiku / Mini does tool selection; Opus / GPT-5.5 does the actual reasoning step.
- **Always have a stopping condition** — Max steps, max cost, success predicate. Infinite loops are real.
- **Log every step** — You will not debug an agent without traces. LangSmith, Weights & Biases Weave, or just structured JSON logs.
- **Cache aggressively** — If two agents in a workflow see the same docs, share the cache prefix.
- **Human-in-the-loop checkpoints** — For anything that writes / sends / spends money, require approval. Same rule as in CI/CD.

9. Your laptop folder structure

Principle

> Keep **identity** (configs, skills, MCP servers), **artifacts** (notes, output PDFs, generated code), and **projects** (per-product work) cleanly separated. Then everything is git-trackable, syncable, and recoverable.

Recommended layout (Mac / Linux; Windows is analogous)

```
~/
  .claude/                          <- Claude Code & Claude.ai config
    skills/                         <- personal skills (git-tracked)
      excel-reports/
      research-summary/
      ...
    commands/                       <- legacy slash commands (still work)
    agents/                         <- sub-agent definitions
    settings.json                   <- editor, theme, defaults
```

```
.claude.json                    <- MCP servers list
.config/
  openai/                       <- OpenAI CLI config
  gemini/                       <- Gemini CLI config

ai/                             <- everything AI-related lives here
  prompts/                      <- reusable prompts library
    writing/
    coding/
    research/
  notes/                        <- AI-generated notes (study, summaries)
    papers/
    meetings/
    research/
  transcripts/                  <- saved conversations worth keeping
  outputs/                      <- generated artifacts (PDFs, docs)
  knowledge/                    <- your RAG corpus (chunked sources)
  experiments/                  <- one-off tests, sandboxes

projects/                       <- real work
  client-acme/
    CLAUDE.md                   <- project-scoped instructions
    .claude/
      skills/                   <- project-scoped skills
    src/
    ...
  side-project-foo/

archive/                        <- old projects (read-only)
scratch/                        <- disposable; nuked weekly
```

Why this works

- **Single source of truth for AI identity** — All your skills, prompts, and configs live under ~/.claude and ~/ai. Easy to git, easy to sync across machines.
- **Projects stay clean** — Per-project skills live inside the repo, not polluting your global config. Other people on the team get them automatically when they clone.
- **Outputs separated from sources** — ~/ai/outputs can be wiped anytime; ~/ai/notes is precious.
- **Easy to backup the precious stuff** — ~/.claude, ~/ai/notes, and ~/ai/prompts are typically <100 MB total. Sync them everywhere.

Git-track these directories

Directory	Why git it	What to .gitignore
~/.claude/skills	Versioned skill changes	anything with secrets
~/ai/prompts	Reusable prompt library	personal/private prompts
~/ai/notes	Knowledge over time	drafts, junk/
projects/*/.claude	Per-repo skills + commands	credentials

■ One-line sync setup

Create a private GitHub repo called *ai-config*. git init ~/.claude && git remote add origin .... Push from your main machine; pull on the laptop. Now your skills, prompts, and MCP setup follow you everywhere.

10. Token & cost optimization

The five-lever ladder

(1) **Model routing** -- Haiku for triage, Opus for hard work. (2) **Prompt caching** -- 90% off cached input. (3) **Batch API** -- 50% off non-realtime. (4) **Context discipline** -- summarize, retrieve, don't dump. (5) **Output discipline** -- ask for what you need, not more.

Lever 1: model routing

The single biggest cost-saver. Use Haiku 4.5 ($1/$5) or GPT-5.2 mini for classification, extraction, summarization, simple routing. Reserve Opus / GPT-5.5 Pro for actual hard reasoning. Cost ratio is often 5-25x.

```
# Example: a 2-stage agent
1. Haiku classifies the user request:
     'code' | 'research' | 'writing' | 'data'
2. Route to specialized prompt + model:
     code     -> Sonnet 4.6
     research -> Sonnet 4.6 + web search
     writing  -> Opus 4.7
     data     -> Sonnet + Code Execution

# Effect on a 100-call workload:
All-Opus:        100 x $0.10 = $10.00
Routed:          100 x $0.01 (Haiku) + 30 x $0.05 (mixed) = $2.50
                 -> 4x cheaper
```

Lever 2: prompt caching

If you call the same model multiple times with a large shared prefix (system prompt, RAG documents, examples), enable prompt caching. After the first call, the prefix costs ~10% of its uncached price. Anthropic, OpenAI, and Google all support this; the markers differ slightly.

```
# Anthropic example (Python SDK)
client.messages.create(
  model='claude-sonnet-4-6',
  system=[
    {'type': 'text',
     'text': LONG_SYSTEM_PROMPT,
     'cache_control': {'type': 'ephemeral'}},  # <-- cached for 5 min
  ],
  messages=[{'role': 'user', 'content': user_q}],
)
```

Lever 3: batch API

If you can wait up to 24 hours for results -- nightly reports, document processing, bulk analysis -- send via the batch API. Half the price. Available across Anthropic, OpenAI, Google.

Lever 4: context discipline

- Don't paste an entire 200-page PDF if you only need the conclusion. Extract the section first.
- Summarize long chats periodically (Claude Code: /compact).
- Use RAG: chunk the corpus, retrieve only the top-k relevant chunks per query.
- For repeated tasks, build a skill -- the SKILL.md is shorter than the ad-hoc instructions you'd otherwise send every time.

Lever 5: output discipline

- Output costs 5x input. Ask for exactly the shape you want.
- Use max_tokens to cap.
- For structured data, use JSON mode -- avoid the model writing a preamble.
- Avoid 'explain in detail' when 'in 3 bullets' would do.

Quick decision table

Situation	Optimal lever
Same system prompt every call	Prompt caching
10,000 docs to classify overnight	Batch + Haiku
Conversational chat, real-time	Streaming + Sonnet + caching
Complex one-off reasoning	Opus with extended thinking, no cache
Routing requests to specialists	Haiku classifier + per-class prompt
Need 1M-token context	Gemini 3.1 Pro or Opus 4.7 (long-context tier)
Heavy code editing in IDE	Cursor/Claude Code with caching

11. Daily / weekly / monthly cadence

Why have a cadence

AI tools evolve fast and your workflow rots faster. A regular rhythm keeps skills sharp, costs visible, and stops drift before it compounds.

Daily (5-10 min)

Time	Task
Start of day	Open your prompt library; pick today's reusable prompts
During work	Pin one Claude / Cursor / IDE session per task; don't tab-juggle
End of session	If you used a new pattern 2+ times, draft a skill for it
End of day	Save any worth-keeping conversation to ~/ai/transcripts/

Weekly (30-60 min)

Block	Task
Review costs	Check API dashboard. Anything > expected? Pinpoint which call.
Skills review	List skills you used; delete unused; refine descriptions of the under-triggering ones
Prompt library	Promote ad-hoc one-liners that worked into ~/ai/prompts/
Commit configs	cd ~/.claude && git add -A && git commit && git push
Try one new thing	A new model release, a new MCP server, or a new framework

Monthly (1-2 hours)

Block	Task
Cost report	Total spend across vendors; which model accounted for what %?
Model audit	Did any tasks regress on a new model release? Should anything migrate?
Skill consolidation	Merge overlapping skills; archive stale ones (move to ~/.claude/skills/_archive/)
MCP audit	Disconnect MCP servers you haven't used; rotate secrets
Knowledge prune	Drop outdated notes from ~/ai/notes/; refresh stale references
Backup verification	Confirm ~/.claude and ~/ai are syncing to remote

Quarterly (half a day)

Block	Task
Vendor review	Has any vendor changed pricing? Should you switch model tiers?
Stack review	Are you still using Cursor / Claude Code / Copilot in the right mix?
Capability check	Run your benchmark prompts against the current top models; record results
Folder hygiene	Re-evaluate the folder structure. Anything outgrown it?
Read one paper or doc	Anthropic / OpenAI / Google all publish; pick one to actually read

A sample personal weekly report template

```
# Week of {DATE}

## Spend
- Anthropic API: $___ (last week: $___)
- OpenAI API:    $___
- Top model:     ___% of spend

## What I built / did
- Project A: ___
- Project B: ___

## New skills / prompts created
- ___

## Pain points
- ___

## Next week experiments
- ___
```

12. Putting it all together

End-to-end mental model

You sit at a terminal or chat UI. **Claude / GPT / Gemini** is the brain. **Skills** tell it how to do recurring tasks. **MCP servers** let it talk to your tools. **The CLI (Claude Code)** coordinates it all. Your **folder structure** keeps configs portable. Your **cadence** keeps it from rotting. Cost is managed by **routing + caching**.

Reference workflow A: write a research summary

```
1. ~/ai/prompts/research/summarize-paper.md  <- your standard prompt
2. Open Claude.ai (or claude in terminal)
3. Drop PDF + prompt -> Sonnet 4.6 produces draft
```

```
4. Skill 'pdf-builder' formats it as a clean PDF (if requested)
5. Save output to ~/ai/notes/papers/{paper-name}.pdf
6. If you used a new pattern -> draft a skill
7. Commit ~/ai/notes weekly
```

Reference workflow B: code on a real project

```
1. cd ~/projects/my-app
2. Project has its own CLAUDE.md + .claude/skills/
3. claude (Claude Code)
4. Mention task; Claude reads CLAUDE.md, fires relevant skill
5. MCP servers (GitHub, Postgres) connect on demand
6. Claude edits files, runs tests, asks for permission on writes
7. /cost at end -> log to weekly report
8. git commit; /compact to save context for next session
```

Reference workflow C: bulk processing

```
1. ~/ai/experiments/bulk-classify/
2. Python script with Anthropic Batch API + Haiku 4.5
3. Submit batch; receive in <24h
4. Process results -> save to ~/ai/outputs/
5. Cost: half of standard, plus 5x cheaper Haiku
6. Total: ~10% of what Opus realtime would have cost
```

When to use which tool

Task shape	Best home
One-off question, no files	Claude.ai / ChatGPT / Gemini app
Multi-file editing in a repo	Claude Code (CLI) or Cursor (IDE)
Inline code completion	Cursor / Copilot
Document drafting + iteration	Claude.ai / Claude Pro
Web research + citations	Perplexity / ChatGPT search
Bulk data processing	API + Batch + Haiku/mini
Real-time conversation	ChatGPT voice / Gemini Live
Whole codebase Q&A	Gemini 3.1 Pro (2M ctx) or Claude Opus 4.7 (1M)
Custom internal tool	MCP server + Claude Code
Self-hosted / private	Llama 4 / Mistral / DeepSeek on your hardware

Final checklist

- ~/.claude and ~/ai exist and are git-tracked
- At least 3 personal skills you actually use
- CLAUDE.md in every active project
- 3-5 MCP servers configured, no more
- Prompt library with categories
- Monthly cost report habit
- Weekly git push of configs

- One vendor's billing alert set above 1.5x your typical spend
- Backup of ~/.claude and ~/ai on a second device or cloud

End of guide. Information current as of May 2026.
Verify pricing on vendor pages before committing to a tier.